Основы глубинного обучения

Лекция 5

Приёмы обучения нейросетей

Евгений Соколов

esokolov@hse.ru

НИУ ВШЭ, 2026

AdamW

- L2-регуляризация: добавляем к функции потерь $\frac{\lambda}{2}\|w\|^2$, в градиенте появляется слагаемое λw:

$$g_t = \nabla Q(w^{t-1}) + \lambda w^{t-1}$$

  - дальше обычный шаг Adam по g_t: штраф делится на $\sqrt{\hat{v}_t}$ вместе с градиентом
  - у параметров с большими градиентами регуляризация почти исчезает
- AdamW (decoupled weight decay): штраф вычитаем из весов отдельно, минуя нормировку:

$$w^t = w^{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda w^{t-1} \right)$$

- Для SGD обе формы совпадают, для Adam — нет

https://arxiv.org/abs/1711.05101

Muon (2024–2025)

- Веса линейного слоя — матрица W, градиент по ним G — тоже матрица. Adam масштабирует каждый элемент G по отдельности и не видит, что это матрица
- Muon берёт градиент с инерцией M_t и заменяет его на ближайшую ортогональную матрицу: все сингулярные числа становятся равны 1

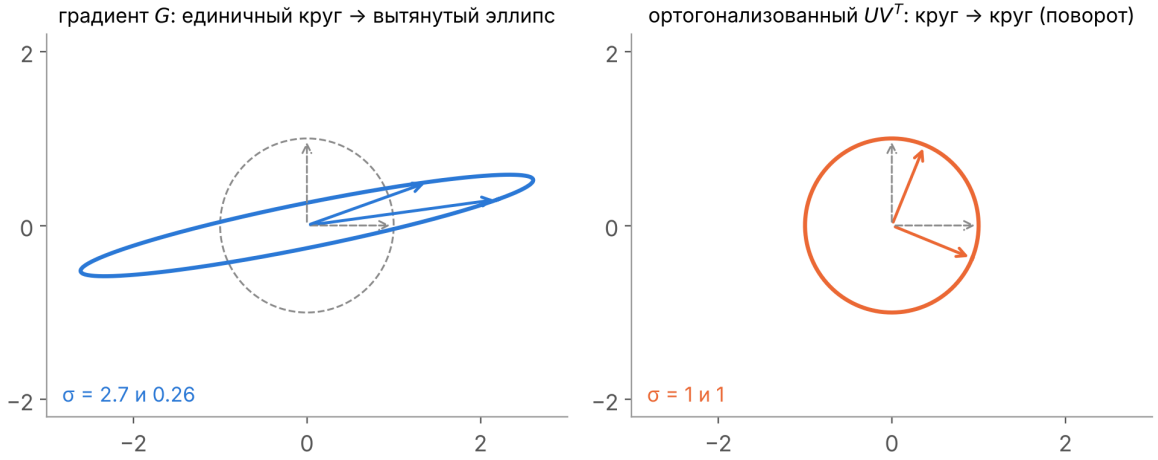
$$W^t = W^{t-1} - \eta_t \cdot UV^T, \qquad \text{где } M_t = U\Sigma V^T$$

  - смысл: обновление одинаково сильно двигает все направления, а не только несколько доминирующих
- Применяется к матрицам внутри сети; эмбеддинги, выходной слой и параметры нормализаций — по-прежнему AdamW
- Результат: до того же лосса примерно вдвое меньше вычислений, чем у AdamW (Moonshot, 2025); Kimi K2 (1 трлн параметров) обучена Muon

https://arxiv.org/abs/2502.16982

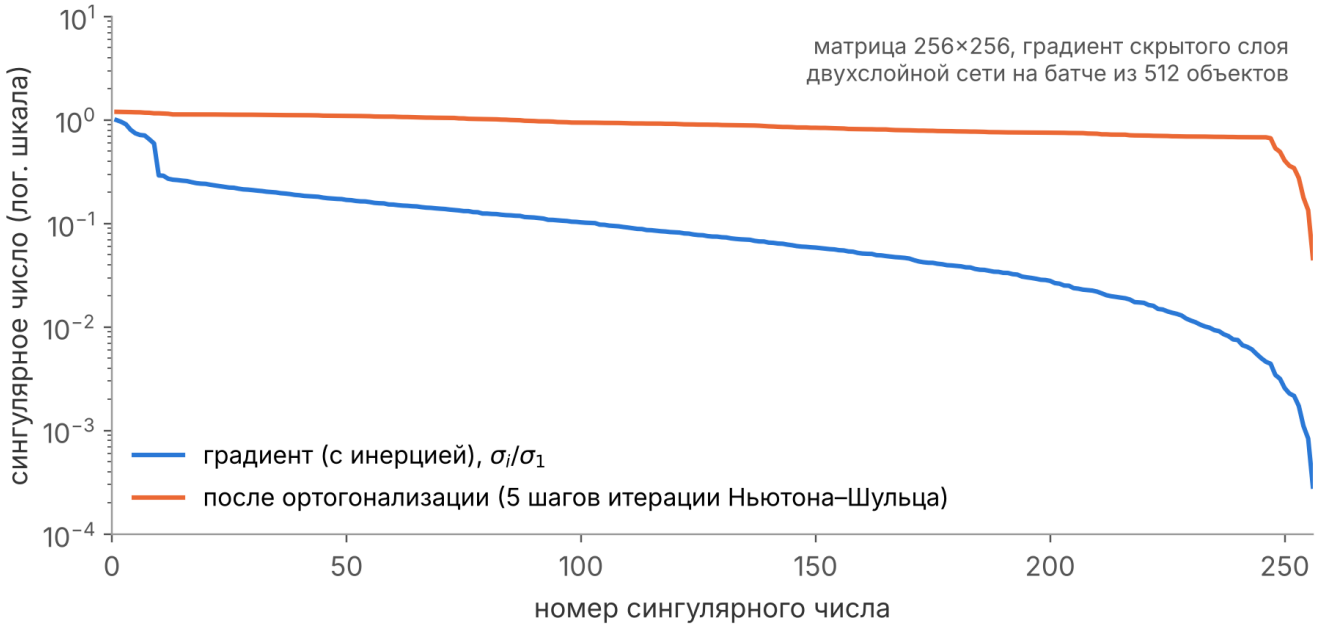
Ортогонализация

- Матрица обновления G — линейное отображение
  - единичный шар она растягивает в эллипсоид
  - полуоси — сингулярные числа σ_i
- Большое σ_i — направление, по которому шаг двигает веса сильно, маленькое — почти не двигает
- Ортогонализация оставляет направления (поворот UV^T) и выравнивает длины: все $\sigma_i = 1$
- Редкие направления получают такой же шаг, как доминирующие



Спектр матрицы производных

- Градиент матрицы весов почти низкоранговый: лишь несколько сингулярных чисел большие
  - на картинке 65% энергии обновления лежит в 10 направлениях из 256
- Adam масштабирует каждый элемент матрицы по отдельности и этого не видит
- Muon воздействует на матрицу целиком



https://kellerjordan.github.io/posts/muon/

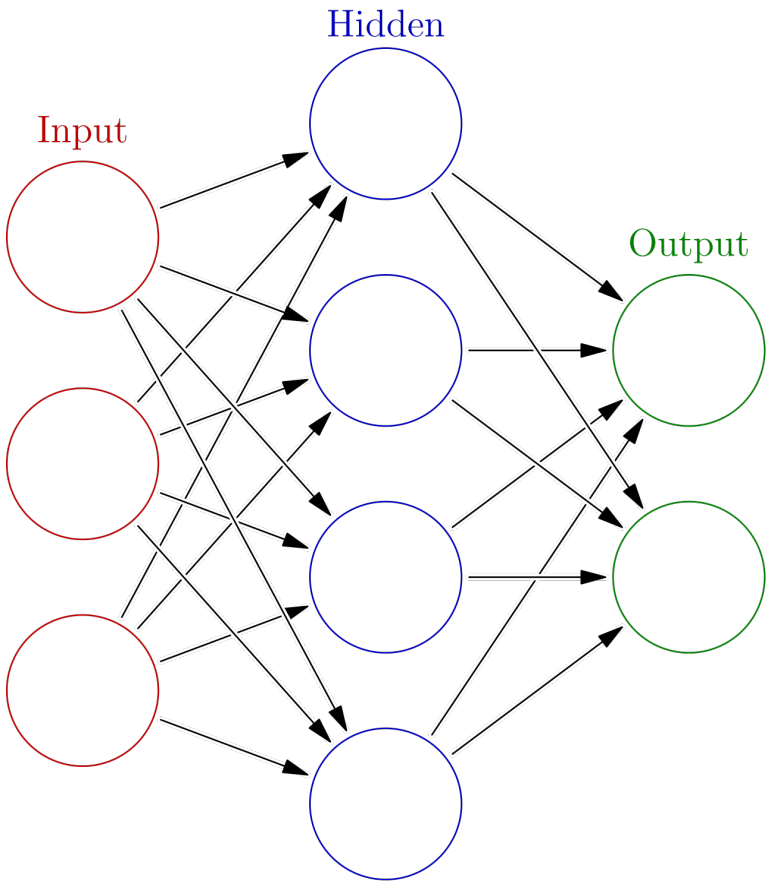
Dropout

Борьба с переобучением

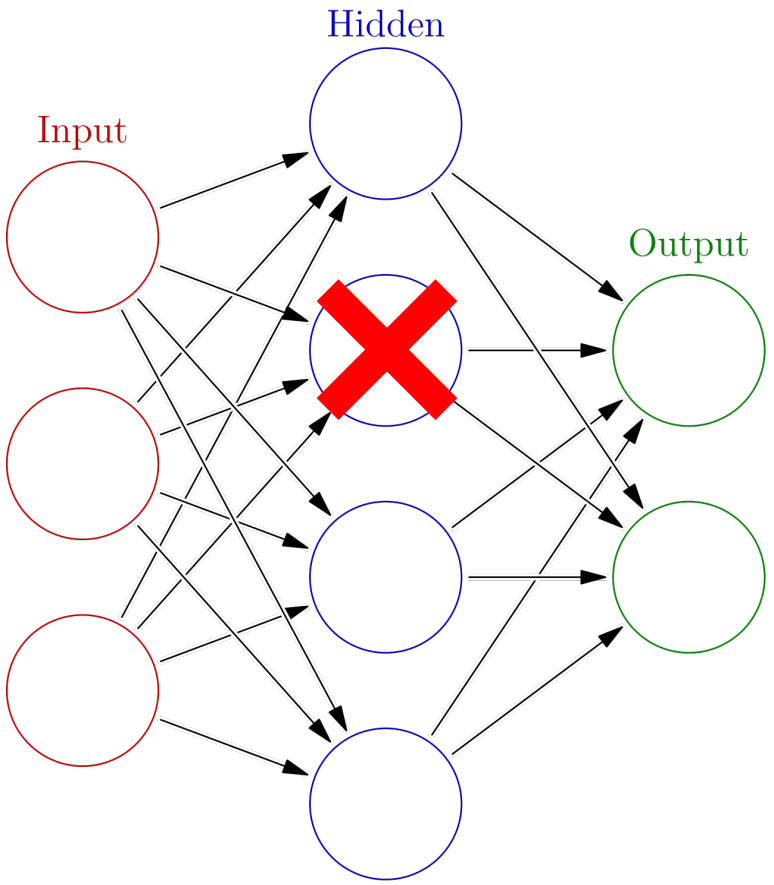
- Сокращение числа параметров (свёрточные слои помогают с этим)
- Регуляризация

- Можно как-то ещё мешать модели подгоняться под обучающую выборку

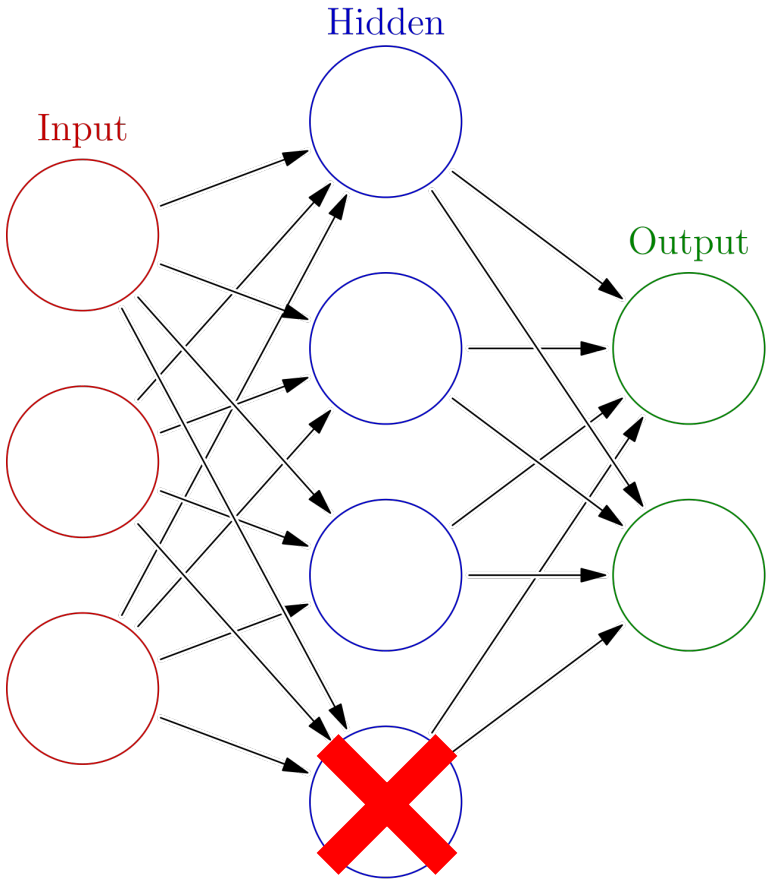
Dropout



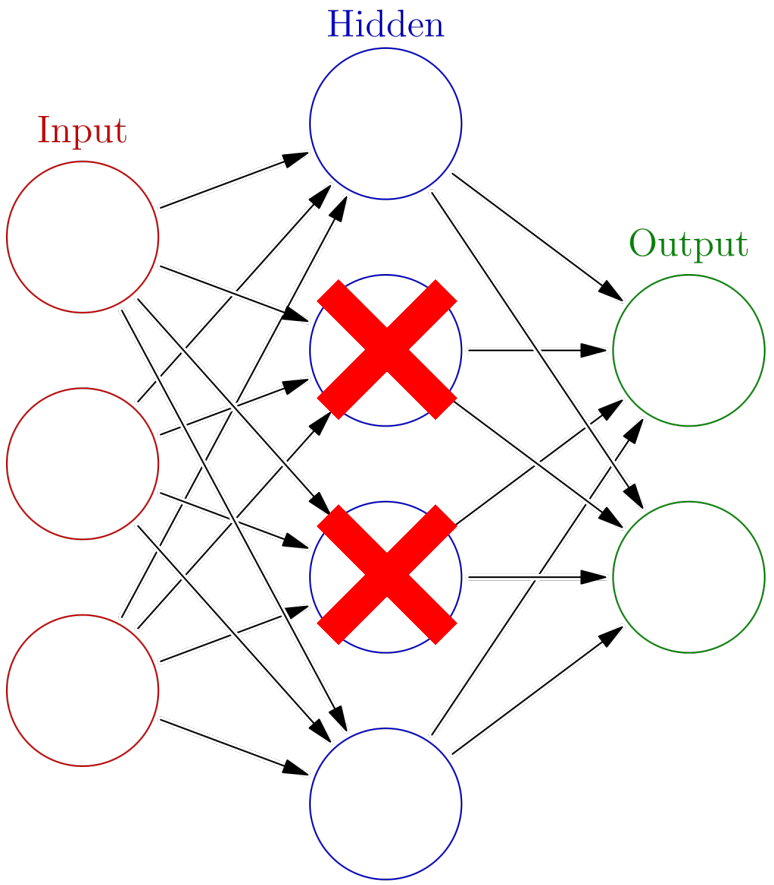
Dropout



Dropout



Dropout



Dropout

- Можно определить как слой $d(x)$
- Параметров нет, единственный гиперпараметр — p (вероятность удаления нейрона)
- На этапе обучения:

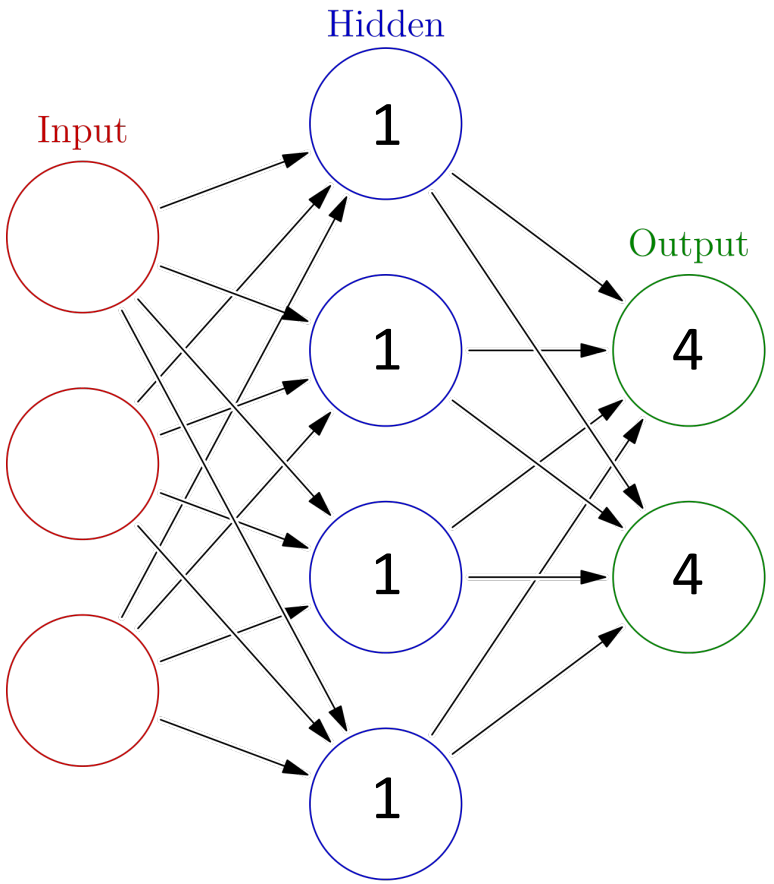
$$d(x) = \frac{1}{1-p} m \circ x$$

- m — вектор того же размера, что и x, элемент берутся из распределения $\mathrm{Ber}(1-p)$
- Деление на $(1-p)$ — для сохранения суммарного масштаба выходов

https://jmlr.org/papers/v15/srivastava14a.html

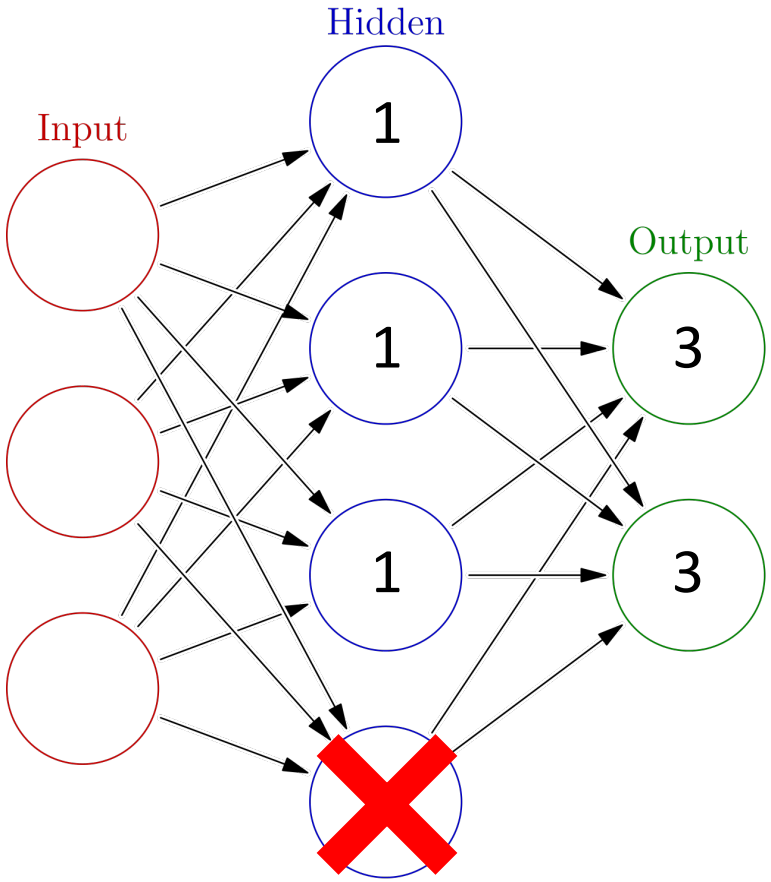
Dropout

Пусть все веса единичные



Dropout

Пусть все веса единичные



Надо компенсировать снижение масштаба суммы выходов!

Dropout

- На этапе обучения:

$$d(x) = \frac{1}{1-p} m \circ x$$

- На этапе применения:

$$d(x) = x$$

В оригинальной статье нет нормировки на этапе обучения, но есть домножение на $(1-p)$ на этапе применения

Вариант на слайде — inverted dropout (чуть меньше операций во время применения сети)

Dropout

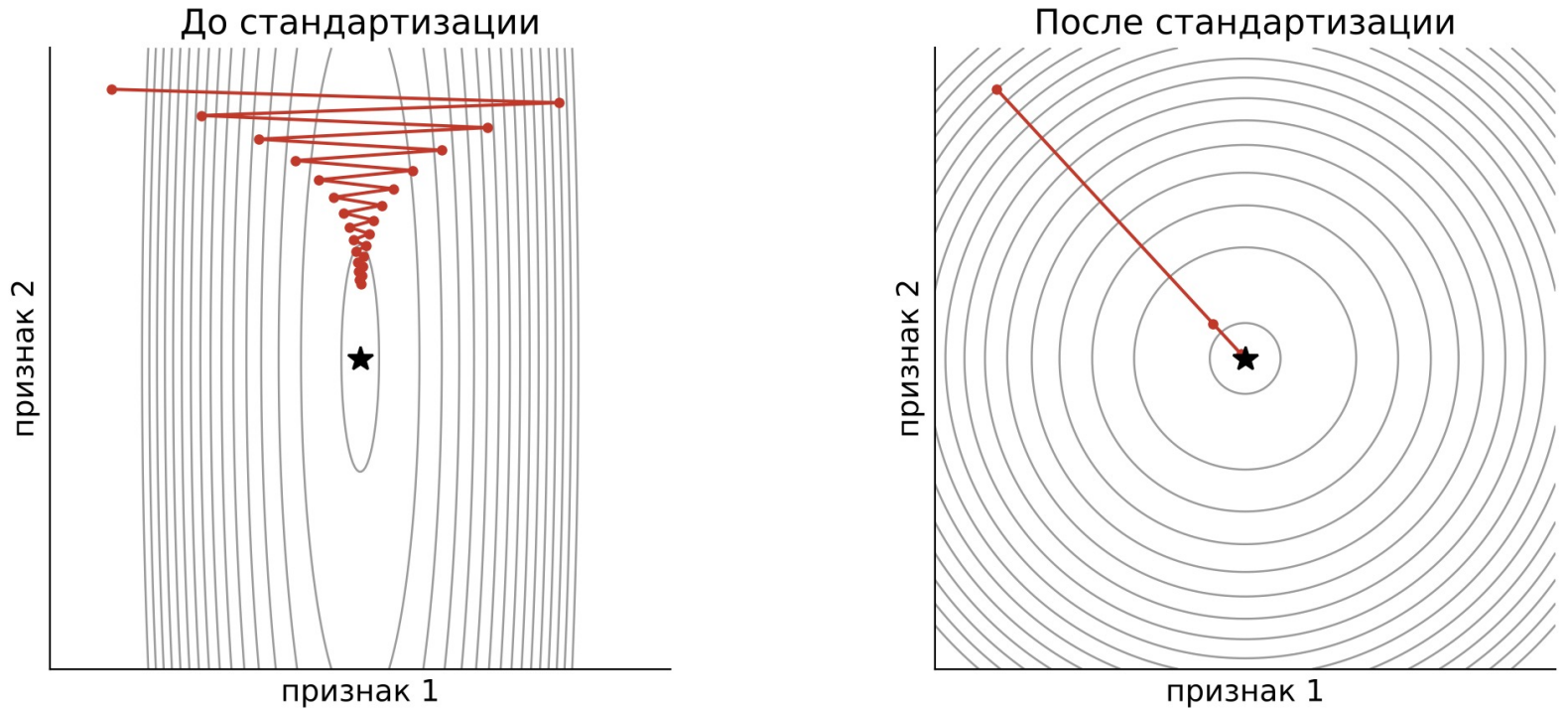
- Интерпретация: мы обучаем все возможные архитектуры нейросетей, которые получаются из исходной выбрасыванием отдельных нейронов
- У всех этих архитектур общие веса
- На этапе применения (почти) усредняем прогнозы всех этих архитектур

- Сейчас: в свёрточных сетях почти не используется, BatchNorm и аугментация работают лучше
- Применяется в полносвязных головах и в трансформерах ($p \approx 0.1$); в предобучении больших языковых моделей обычно не используется: данных больше, чем эпох, переобучаться не на чем

Нормализации

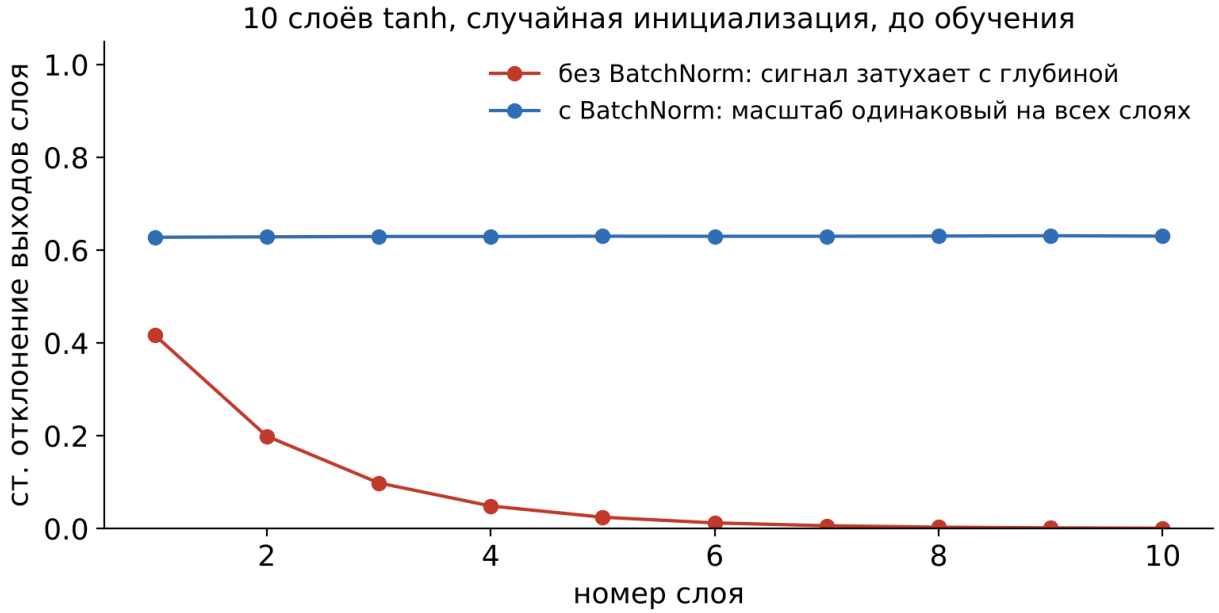
Масштабирование признаков

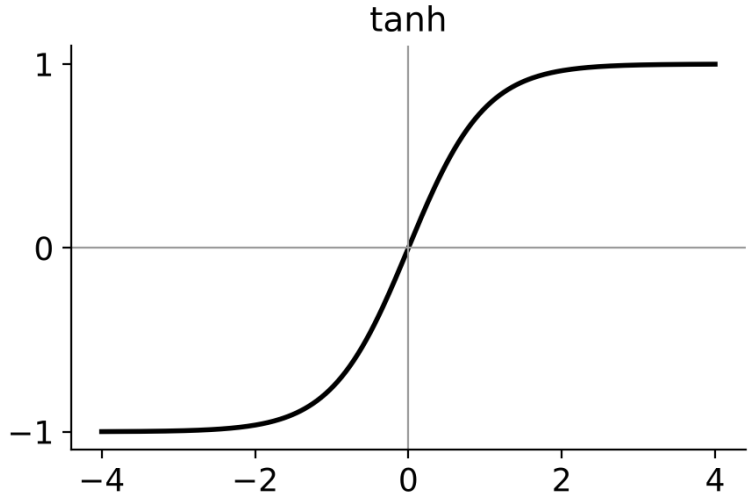
- В классическом ML признаки принято масштабировать
- Каждый слой нейросети — модель над выходами предыдущего
- Почему бы не масштабировать вход каждой модели?



Что происходит без масштабирования

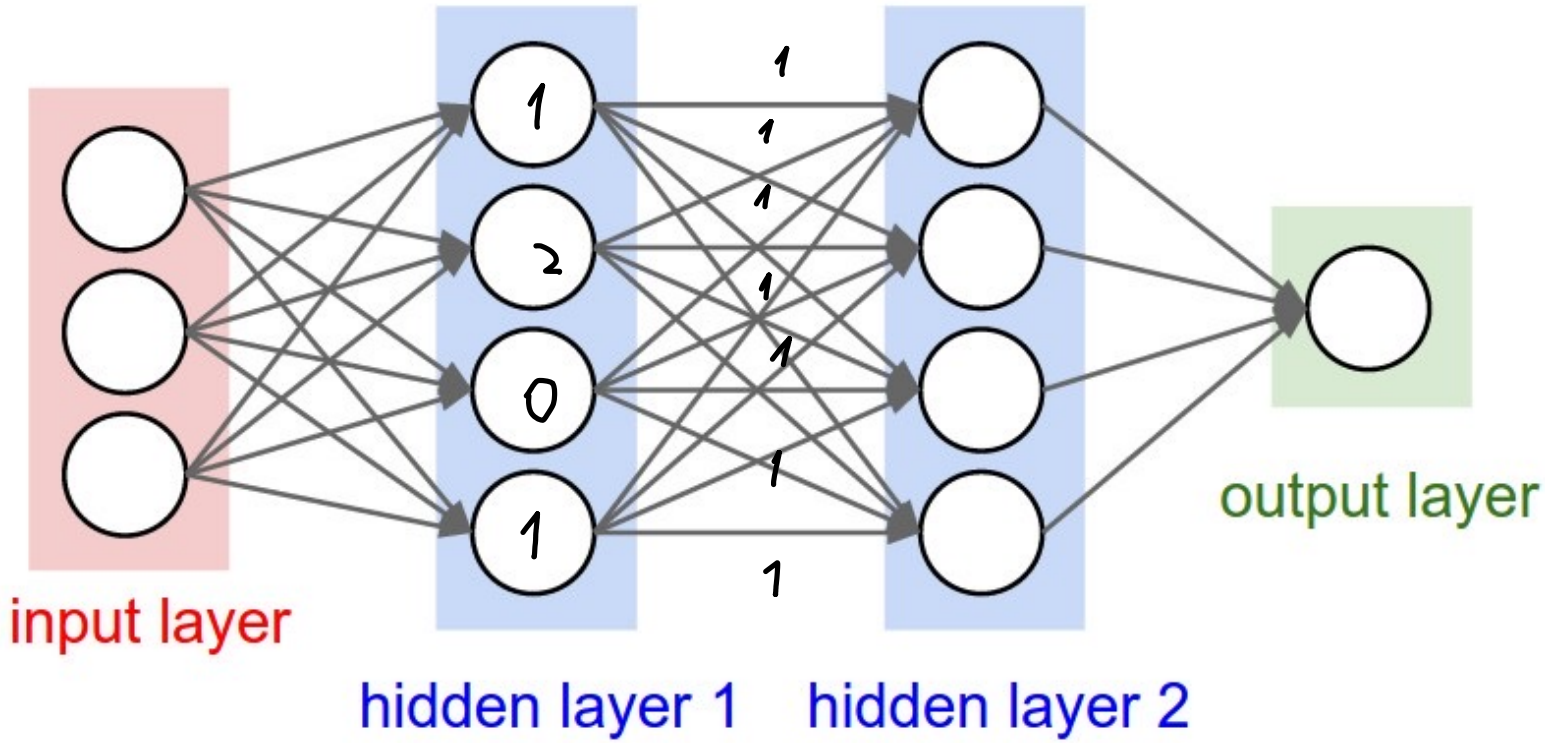
- Масштаб выходов меняется от слоя к слою: при неудачной инициализации сигнал затухает или насыщает нелинейность
- Градиент по весам слоя пропорционален его входам: $\frac{\partial L}{\partial W} = \delta \cdot x^T$. Где входы в 100 раз меньше, там при той же η шаг в 100 раз меньше
- Одна длина шага на все слои не подходит, пока входы у всех в разном масштабе





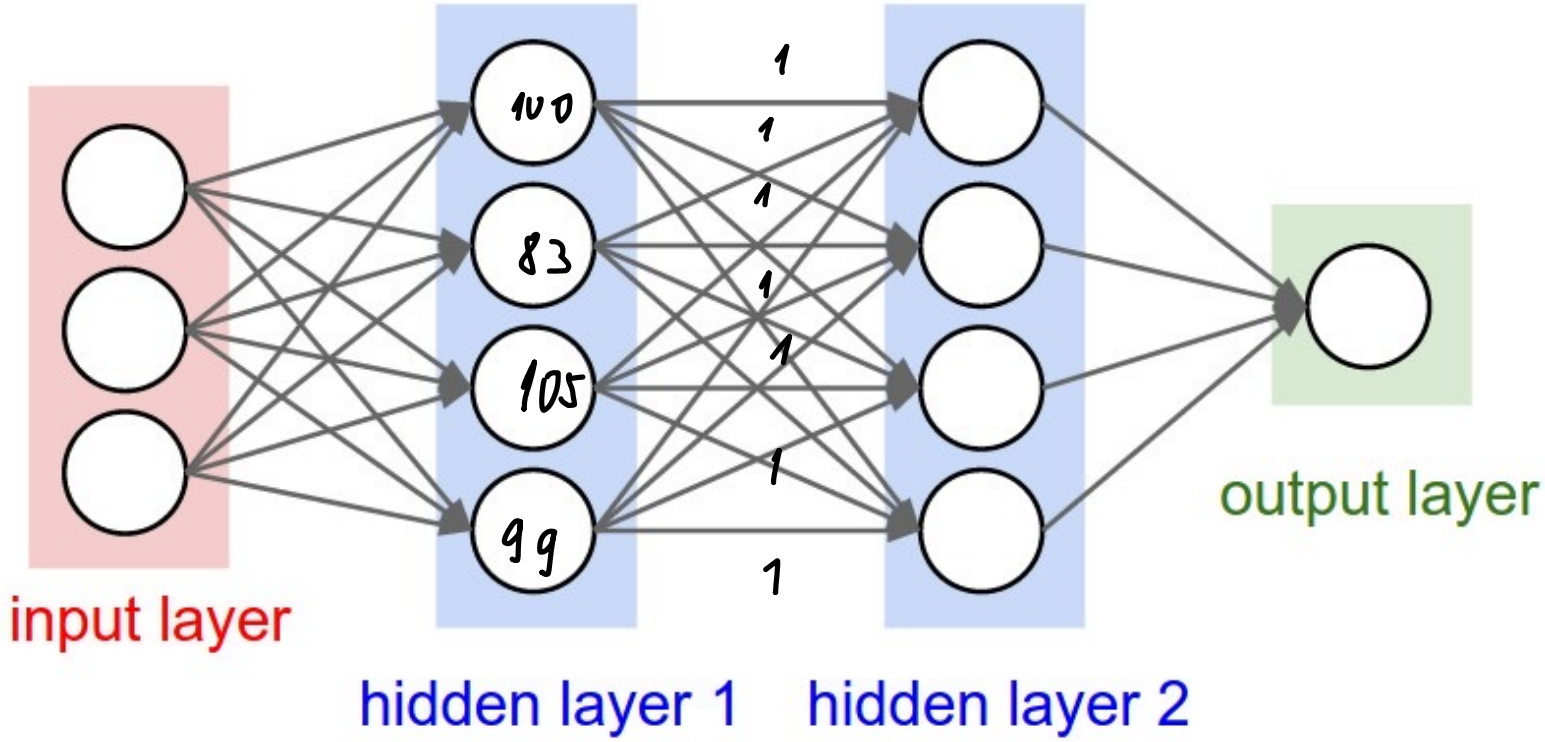
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Internal covariate shift



Internal covariate shift

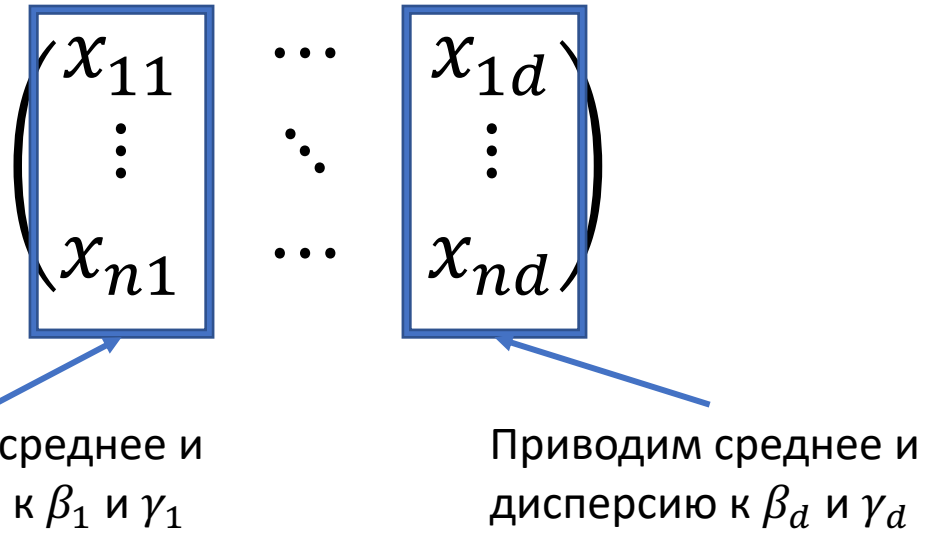
Допустим, веса первого слоя сильно поменялись после градиентного шага



Идея

- Входы слоя зависят от весов предыдущих слоёв, а они меняются на каждом шаге
- Пересчитывать по всей выборке на каждом шаге — дорого
- Выход: считать среднее и дисперсию по текущему батчу — он всё равно уже у нас в руках
- Авторы (Ioffe, Szegedy, 2015) назвали проблему internal covariate shift — но ещё обсудим, так ли это

Batch Normalization



- n — размер батча
- d — размерность входного вектора

Пример на числах

- Батч из 4 объектов, 3 признака
- Масштабируем каждый столбец со своими μ и σ

Батч на входе слоя

	рост	стаж	возраст
объект 1	180	2	30
объект 2	165	0.5	45
объект 3	172	8	22
объект 4	158	1.2	61

μ = 2.92

σ = 2.98

$(x - \mu) / \sigma$
по столбцам



После BatchNorm

рост	стаж	возраст
+1.38	-0.31	-0.64
-0.46	-0.81	+0.37
+0.40	+1.70	-1.17
-1.32	-0.58	+1.44

каждый столбец: μ = 0, σ = 1

Batch Normalization

- Реализуется как отдельный слой
- Вычисляется для текущего батча
- Оценим среднее и дисперсию каждой компоненты входного вектора:

$$\mu_B = \frac{1}{n}\sum_{j=1}^{n} x_{B,j}$$

покоординатно

$$\sigma_B^2 = \frac{1}{n}\sum_{j=1}^{n}\left(x_{B,j} - \mu_B\right)^2$$

$x_{B,j}$ — j-й объект в батче B

Batch Normalization

- Отмасштабируем все выходы:

$$\tilde{x}_{B,j} = \frac{x_{B,j} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

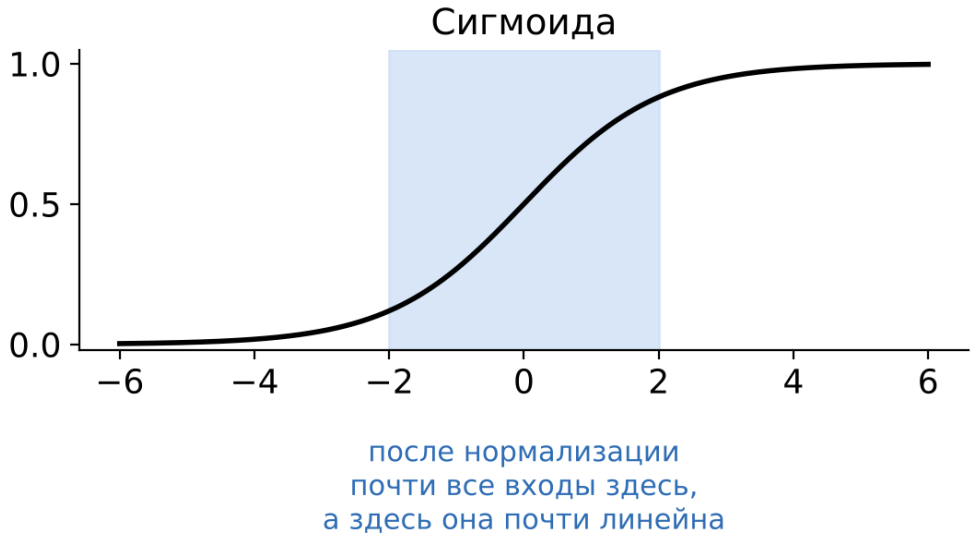
- Зададим нужные нам среднее и дисперсию:

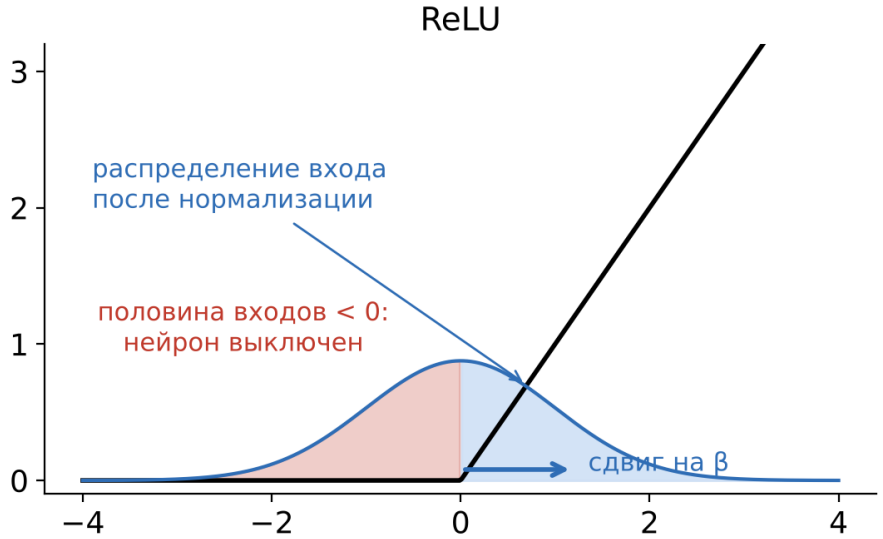
$$z_{B,j} = \gamma \circ \tilde{x}_{B,j} + \beta$$

обучаемые параметры (векторы, размерность равна размерности входных векторов)

Зачем γ и β

- Без них выход каждого нейрона имеет среднее 0 и дисперсию 1 — а это не всегда то, что нужно следующему слою
  - сигмоида: входы почти всегда в [−2, 2], где она почти линейна — нелинейность пропадает
  - ReLU: среднее 0 → половина входов отрицательна, для каждого объекта половина нейронов выключена; β сдвигает порог, а γ для ReLU почти не важна: $\mathrm{ReLU}(\gamma x) = \gamma \cdot \mathrm{ReLU}(x)$
- γ и β обучаемые: сеть сама выбирает масштаб и сдвиг каждого выхода; может даже отменить нормализацию ($\gamma = \sigma, \beta = \mu$)
- Среднее и дисперсия стали явными параметрами





Batch Normalization

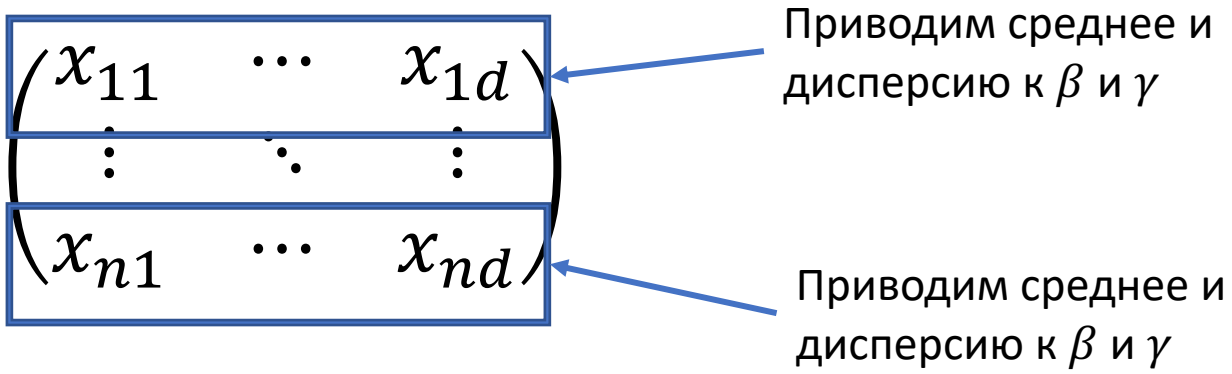
Важно: после BatchNorm среднее и дисперсия каждого выхода зависят только от параметров нормализации, но не от параметров прошлых слоёв!

Batch Normalization

Во время применения нейронной сети:

- Те же самые формулы, но вместо μ_B и σ_B^2 используем их средние значения по всем батчам

Layer Normalization



- n — размер батча
- d — размерность входного вектора
- Нормируем внутри одного объекта, по его признакам: от размера батча не зависит, на обучении и применении работает одинаково

https://arxiv.org/pdf/1607.06450.pdf

Layer Normalization

- Нормализуем распределение «признаков» одного объекта

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij}$$

$$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}\left(x_{ij} - \mu\right)^2$$

x_{ij} — j-й признак i-го объекта

https://arxiv.org/pdf/1607.06450.pdf

Layer Normalization

- Отмасштабируем все выходы:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

- Зададим нужные нам среднее и дисперсию:

$$z_{ij} = \gamma \circ \tilde{x}_{ij} + \beta$$

обучаемые параметры — векторы размерности d: свои gain и bias у каждого нейрона

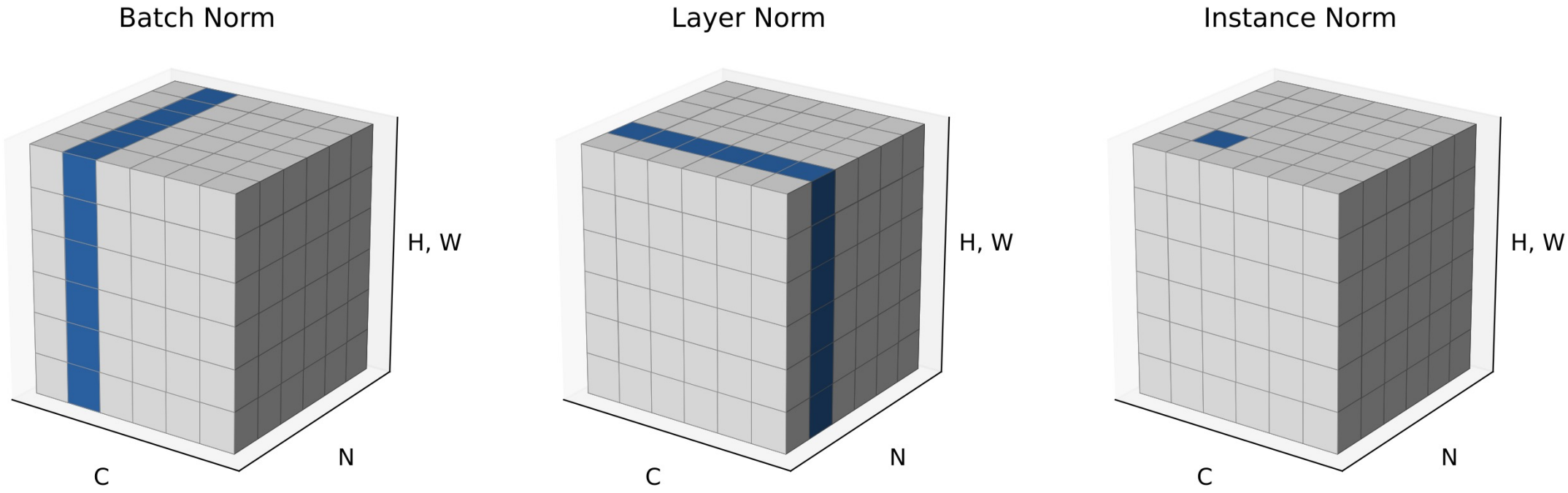
https://arxiv.org/pdf/1607.06450.pdf

BatchNorm для свёрточных слоёв

- Вход — тензор $N \times C \times H \times W$
  - N — объектов в батче, C — каналов, H и W — высота и ширина карты
- Статистики считаются на канал: усредняем по объектам и по всем позициям, то есть по осям N, H, W
  - на каждый канал — одно μ, одно σ^2, одна пара γ и β; итого $2C$ обучаемых параметров
- Почему так: свёртка одна и та же для всех позиций карты, значит и масштабирование канала должна быть одинаковой во всех позициях

По каким осям усредняем

- Batch Norm: по объектам и по позициям — одно μ и σ на канал
- Layer Norm: по каналам и позициям — одно μ и σ на объект
- Instance Norm: только по позициям — на объект и канал

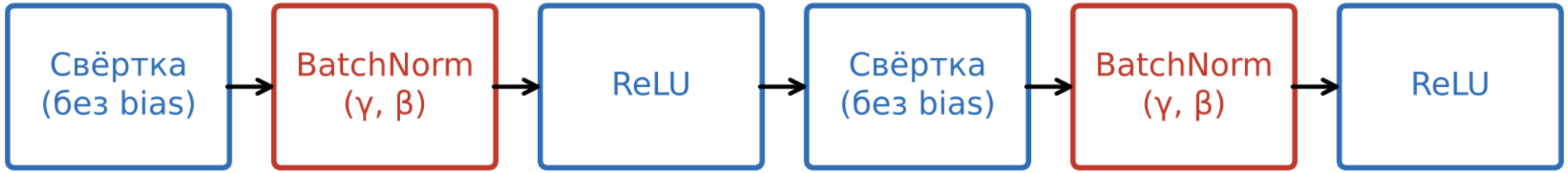


Синие ячейки нормируются одним и тем же μ и σ, посчитанными по этим же ячейкам

Batch Normalization

- Обычно вставляется между полносвязным/свёрточным слоём и нелинейностью
- Позволяет увеличить длину шага в градиентном спуске

- Не факт, что действительно устраняет covariate shift



bias перед BatchNorm бесполезен: вычитание среднего его обнуляет, за сдвиг отвечает β

В чём польза от BatchNorm?

How Does Batch Normalization Help Optimization?

Shibani Santurkar* MIT shibani@mit.edu

Dimitris Tsipras* MIT tsipras@mit.edu

Andrew Ilyas* MIT ailyas@mit.edu

Aleksander Mądry MIT madry@mit.edu

https://arxiv.org/pdf/1805.11604.pdf